

1. Project Overview

This project implements a human vs. computer Rock-Paper-Scissors game in C++. The computer supports two selectable algorithms: Random and Smart. Algorithm selection is done before gameplay using command-line options. The default game length is 20 rounds, and both modes share the same rounds configuration variable. The implementation uses both Strategy and Template Method design patterns for full-credit architectural goals.

2. Iterative Development Summary

Iteration 1: Basic playable game

- Implemented core game flow for 20 rounds.
- Implemented human input, round winner evaluation, and score tracking.
- Delivered a complete playable baseline.

Iteration 2: Strategy pattern integration

- Introduced abstract Strategy interface.
- Implemented RandomStrategy.
- Refactored ComputerPlayer to hold a polymorphic strategy object.
- Result: algorithm selection became replaceable without changing engine logic.

Iteration 3: Template Method refactor

- Refactored GameEngine into an abstract class with a fixed playGame skeleton.
- Created ConsoleGameEngine to provide concrete output behavior through hooks.
- Result: core game flow and display logic were separated.

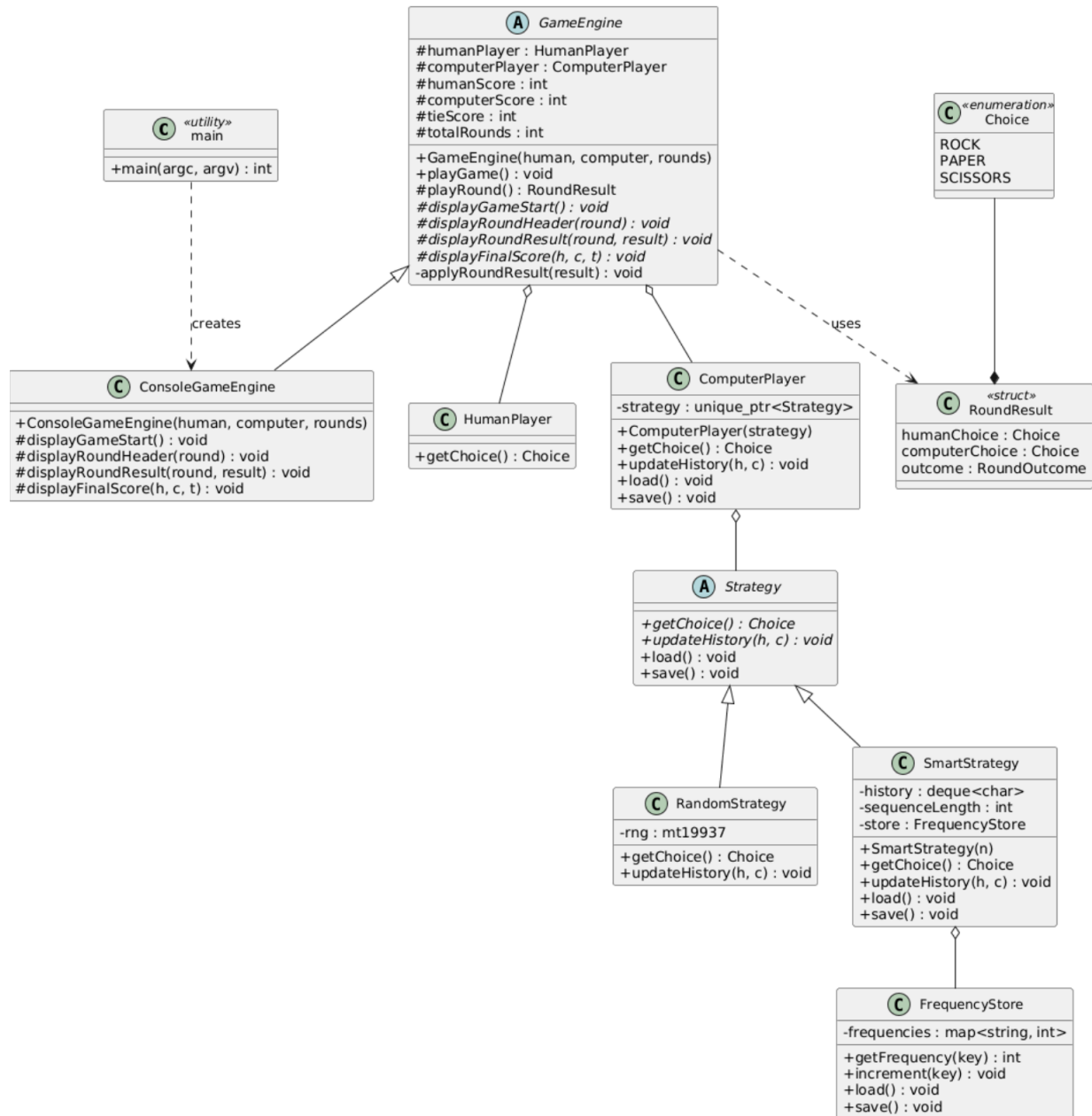
Iteration 4: Smart algorithm and persistence

- Implemented SmartStrategy with sequence-frequency prediction and random fallback.
- Implemented FrequencyStore for frequency map management and disk persistence.
- Added CLI handling and validation for mode selection and smart parameter N.
- Result: smart mode supports cross-game learning through persistent data.

Backtracking assessment

No significant backtracking occurred. Changes were incremental and built on previous iterations. A small forward-compatible adjustment was adding strategy lifecycle methods for loading and saving data, which reduced future rework.

3. UML Class Diagram



4. Class Relationships and Naming Quality

4.1 Relationship quality

- Inheritance:

- Strategy is the abstract base for RandomStrategy and SmartStrategy.
 - GameEngine is the abstract base for ConsoleGameEngine.
- Aggregation/composition:
 - ComputerPlayer owns a strategy object through polymorphism.
 - SmartStrategy owns FrequencyStore for persistent learning data.
 - GameEngine coordinates HumanPlayer and ComputerPlayer.
- Dependency:
 - Main creates and wires concrete objects based on CLI options.

4.2 Naming quality of classes, member functions, and member variables

Class naming is explicit and responsibility-driven:

- FrequencyStore clearly indicates storage and persistence responsibility.
- SmartStrategy and RandomStrategy clearly identify algorithm behavior.
- ConsoleGameEngine clearly identifies console-specific rendering.

Member function naming is intention-revealing:

- parseCliConfig expresses the parsing plus validation role.
- displayRoundResult and displayFinalScore clearly describe output hooks.
- updateHistory, load, and save clearly describe the strategy lifecycle behavior.

Member variable naming is concise and descriptive:

- humanScore, computerScore, and ties clearly represent the scoreboard state.
- sequenceLength and history clearly represent a smart prediction state.
- rounds and smartN clearly represent runtime configuration.

5. Design Principles Used

Single Responsibility

- FrequencyStore handles only frequency data and file I/O.
- HumanPlayer handles only input collection and validation.
- SmartStrategy handles prediction logic only.

Loose Coupling

- Game logic depends on abstractions rather than concrete strategy types.
- Strategy substitution does not require engine changes.

Coding to the Interface

- ComputerPlayer uses Strategy abstraction for all algorithm operations.

Encapsulate What Varies

- Algorithm behavior varies and is encapsulated inside Strategy subclasses.

- Output behavior varies and is encapsulated in ConsoleGameEngine hook overrides.

Hidden Implementation

- Frequency map internals are private in FrequencyStore.
- Prediction state internals are private in SmartStrategy.

Open/Closed Principle

- New algorithm strategies can be added without modifying the engine core flow.

6. Design Patterns Applied

Strategy Pattern

Intent: allow interchangeable algorithms for computer move selection.

Implementation: ComputerPlayer delegates getChoice and updateHistory to injected Strategy implementations (RandomStrategy or SmartStrategy).

Benefit: adding a new strategy requires minimal impact on existing classes.

Template Method Pattern

Intent: keep game flow fixed while allowing customizable steps.

Implementation: GameEngine defines a fixed playGame skeleton; ConsoleGameEngine overrides display hooks.

Benefit: game loop and scoring are stable, while presentation can vary independently.

7. Smart Mode Implementation and Learning Lifecycle

Prediction logic

- Smart mode uses sequence-frequency matching to estimate likely human behavior.
- It selects the counter-move to the most likely predicted human move.
- If insufficient data exists, it safely falls back to random selection.

Persistence lifecycle

1. At game start, historical frequency data is loaded.
2. After each round, the recent sequence history is updated, and frequency counts are incremented.
3. At game end, updated frequencies are saved to disk.

This lifecycle enables cross-game accumulation of behavior patterns and supports progressive improvement over time.